A. D. PATEL INSTITUTE OF TECHNOLOGY
Department of Information Technology

P R O J E C T R E P O R T

Aadhaar / PAN / Mobile Number Validator

Using HTML, CSS, JavaScript and Regular Expressions

Project Type	Web Development Mini Project
Technologies	HTML5, CSS3, JavaScript
Core Concept	Client-side input validation using regular expressions
Prepared By	Pratham Tank - 12502080603015 Mahi Patel - 12502080603022
Department	Information Technology
Institute	A. D. Patel Institute of Technology, CVM University

Table of Contents

1. Abstract..... 3
2. Introduction..... 3
3. Problem Statement..... 3
4. Objectives..... 3
5. Scope of the Project..... 4
6. Tools and Technologies..... 4
7. System Requirements..... 4
8. System Design and Working..... 5
9. Implementation Details..... 5
10. Regular Expressions Used..... 7
11. Testing..... 8
12. Results and Discussion..... 8
13. Advantages..... 9
14. Limitations and Security Considerations..... 9
15. Future Enhancements..... 9
16. Conclusion..... 9
17. References..... 10

1. Abstract

This report describes a small web application that checks whether an Indian mobile number, a Permanent Account Number (PAN) and an Aadhaar number are written in the expected format. The page is built with HTML for structure, CSS for presentation and JavaScript for the checking, which is done with regular expressions. All validation happens in the browser, so the values typed into the form are not sent to a server. The project covers form handling, pattern matching, conditional logic and user feedback in one practical exercise. It checks format only and does not verify that a number has been issued to anyone.

2. Introduction

Web forms ask for contact and identity details all the time, and people mistype them all the time. A mobile number with nine digits, a PAN with a digit where a letter should be, or an Aadhaar number with one digit missing is easy to catch in the browser and expensive to clean up later in a database.

This project is a single-page form with fields for a mobile number, a PAN and an Aadhaar number. When the form is submitted, the script tests all three values and shows either a list of errors or a success message. Nothing else happens: there is no server, no storage and no external service.

3. Problem Statement

Checking mobile and identity-number formats by hand is slow and easy to get wrong, and badly formatted values that reach a system cause failed lookups, rejected submissions and extra cleanup. A lightweight client-side validator is needed that tests the common input patterns and tells the user at once what is wrong.

4. Objectives

The objectives of the project are:

- to build a simple, user-friendly web form with three input fields and one submit button;
- to validate a 10-digit Indian mobile number with a regular expression;
- to validate the standard ten-character PAN format;
- to validate the basic 12-digit Aadhaar format;
- to show a clear error message for each failed field, or a success message when all three pass;
- to practise HTML form handling, CSS styling, JavaScript events and regular expressions.

5. Scope of the Project

The project is meant for learning and demonstration. It checks the format of values typed into a browser form and validates them when the form is submitted.

It does not:

- contact government databases or any other external service;
- confirm that a number has been issued, or that it belongs to a particular person;
- calculate the checksum of an Aadhaar number;
- store, log or transmit the values entered.

6. Tools and Technologies

- HTML5: defines the form, labels, input fields, submit button and result area.
- CSS3: styles the page, form controls and layout, and colors the validation messages.
- JavaScript: reads the field values, handles the form submission, tests the regular expressions and displays the result.
- Regular expressions: define the accepted character pattern for each field.
- Web browser: Chrome, Firefox, Edge or Safari runs the application locally. No server is needed.
- Text editor: Visual Studio Code or Notepad was used to write the code.

7. System Requirements

Hardware requirements

A basic desktop or laptop computer that can run a modern web browser.

Software requirements

A modern web browser, a text editor such as Visual Studio Code or Notepad, and an operating system that supports both.

8. System Design and Working

Figure 1 shows what the page does each time the form is submitted.

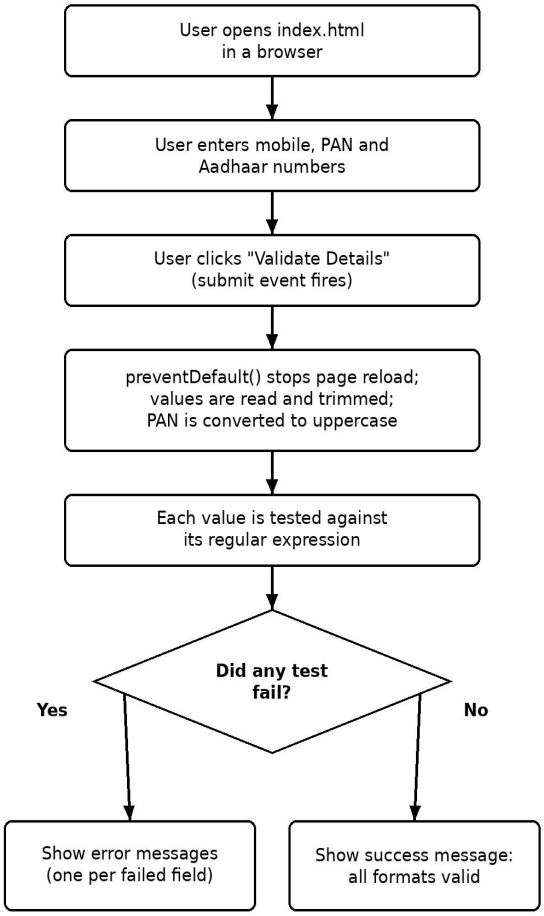


Figure 1: Flow of the validation process

Two design decisions are worth noting.

- Validation runs when the form is submitted, not on every keystroke. This keeps the script short and avoids showing errors while the user is still typing.
- All errors are collected in an array before anything is displayed. A user with two wrong fields sees both messages at once instead of fixing them one at a time.

9. Implementation Details

9.1 HTML structure

The form (id `validatorForm`) contains three text inputs. Each has a label, a placeholder showing the expected shape, the `required` attribute and a `maxlength`. A `div` with `role="status"` and `aria-live="polite"` receives the result, so screen readers announce it when it changes. Table 1 lists the fields.

Table 1: Form fields

Field	Element id	maxlength	Placeholder
Mobile Number	`mobile`	10	Enter 10-digit mobile number
PAN Card Number	`pan`	10	ABCDE1234F
Aadhaar Number	`aadhaar`	12	Enter 12-digit Aadhaar number

9.2 CSS styling

The page has a light grey background and a white card, centered and at most 450 px wide, with rounded corners and a soft shadow. Inputs get a blue outline when focused. The result text is green or red depending on one of two classes, `.success` and `.error`. `box-sizing: border-box` is set on every element so that padding does not push the inputs past the edge of the card.

9.3 JavaScript logic

A submit listener is attached to the form. It stops the default submission, reads the field values and tests each one against its pattern. Failed checks are collected in an error list; if the list is empty, a success message is shown. Listing 1 is the core of the script, shortened where the same block repeats. The bracketed numbers refer to the annotations that follow it.

```
const mobileRegex  = /^[6-9]\d{9}$/;                  // [1]
const panRegex     = /^[A-Z]{5}[0-9]{4}[A-Z]$/;       // [2]
const aadhaarRegex = /^[2-9]\d{11}$/;                 // [3]

form.addEventListener("submit", function (event) {
  event.preventDefault();                             // [4]

  const mobile  = document.getElementById("mobile").value.trim();
  const pan     = document.getElementById("pan")
                    .value.trim().toUpperCase();      // [5]
  const aadhaar = document.getElementById("aadhaar").value.trim();
  const errors = [];

  if (!mobileRegex.test(mobile)) {
    errors.push("Invalid Indian mobile number.");
  }
  // PAN and Aadhaar are tested the same way          // [6]

  if (errors.length > 0) {
    result.className = "error";
    result.innerHTML = errors.join("<br>");           // [7]
  } else {
    result.className = "success";
    result.textContent = "All three details have valid formats!";
  }
});
```

Listing 1: Core validation logic

Annotations for Listing 1:

[1] to [3] The three patterns, explained in Section 10.

[4] Without this call the browser would submit the form in the normal way, reload the page and wipe the result.

[5] `trim()` removes stray spaces. `toUpperCase()` lets a PAN typed in lowercase pass. Mobile and Aadhaar values are digits, so case does not apply to them.

[6] Two more `if` blocks of the same form push "Invalid PAN card format." and "Invalid Aadhaar number format." into the list.

[7] `innerHTML` is used so that `<br>` can separate the messages. This is safe here because the strings are fixed text written by the program, and what the user types is never inserted into the page. The success message uses `textContent`.

10. Regular Expressions Used

Table 2 gives the three patterns, and Figure 2 breaks the PAN pattern into its parts.

Table 2: Regular expressions used in the validator

Field	Regular expression	Meaning
Mobile number	`/^[6-9]\d{9}$/`	Ten digits in total; the first is 6, 7, 8 or 9
PAN number	`/^[A-Z]{5}[0-9]{4}[A-Z]$/`	Five uppercase letters, four digits, one uppercase letter
Aadhaar number	`/^[2-9]\d{11}$/`	Twelve digits; the first is 2 to 9

The pieces used in these patterns:

- `^` and `$` mark the start and end of the input, so the whole value has to match and not just part of it.
- `[6-9]` matches one character from the range, here a single digit.
- `\d{9}` matches exactly nine digits.
- `[A-Z]{5}` matches exactly five capital letters.

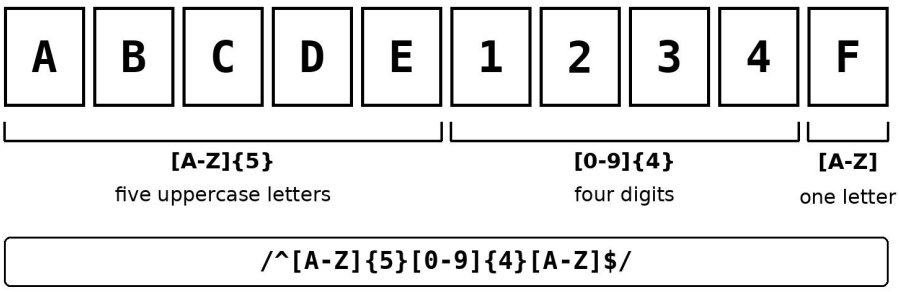


Figure 2: Breakdown of the PAN pattern using the sample ABCDE1234F

Note: Indian mobile numbers begin with 6, 7, 8 or 9, and the Aadhaar pattern follows the rule that a number does not begin with 0 or 1. That is why both patterns start with a restricted range instead of `\d`.

11. Testing

Table 3 lists the test cases. The outcome of each was obtained by running the three patterns, with the same trimming and uppercase step as the program, against the sample input.

Table 3: Test cases and outcomes

No.	Test	Sample input	Result	Reason
1	Valid mobile	`9876543210`	Pass	10 digits, begins with 9
2	Invalid mobile	`1234567890`	Fail	Begins with 1, outside 6-9
3	Invalid mobile	`5876543210`	Fail	Begins with 5, outside 6-9
4	Valid PAN	`ABCDE1234F`	Pass	Letters, digits and letter in the right places
5	PAN in lowercase	`abcde1234f`	Pass	Converted to uppercase before the test
6	Invalid PAN	`AB12345678`	Fail	Digits where letters are required
7	Invalid PAN	`ABCDE12345`	Fail	Last character must be a letter
8	Valid Aadhaar	`234567890123`	Pass	12 digits, begins with 2
9	Invalid Aadhaar	`123456789012`	Fail	Begins with 1
10	Short Aadhaar	`23456789012`	Fail	Only 11 digits
11	Empty field	`(blank)`	Blocked	The required attribute stops submission

Points observed during testing:

- When one field fails, the other two are still checked and reported.
- An empty field never reaches the script, because the browser blocks the submit first.
- The `maxlength` attributes already stop most over-long input from being typed. The `^` and `$` anchors still protect the check if a longer value arrives some other way.

12. Results and Discussion

The validator behaves as intended for every test case in Table 3. A passing result means only that the text has the right shape. Three examples show how much the patterns do not know.

- PAN: in issued PANs the fourth character indicates the type of holder, such as an individual or a company. The pattern accepts any letter in that position.

- Aadhaar: the last digit is a check digit calculated with the Verhoeff algorithm. This program does not calculate it, so a 12-digit number that starts with 2 to 9 passes even if the check digit is wrong.
- Mobile: the pattern cannot tell whether a number is active or assigned to anyone.

For this reason the success message says "valid formats", which is accurate. It would not be accurate to say "valid numbers".

13. Advantages

- Feedback is immediate, with no request to a server.
- The implementation uses only built-in JavaScript features.
- The code is short, and easy to understand, test and maintain.
- It reduces common formatting mistakes in form input.
- More fields and rules can be added by writing one more pattern and one more check.

14. Limitations and Security Considerations

- Regex validation checks format only. It cannot authenticate identity or confirm government records.
- A value that matches a pattern may not be an active mobile number, an issued PAN or an issued Aadhaar number.
- No Aadhaar checksum or UIDAI verification is performed.
- Client-side checks can be bypassed by anyone who edits the page or sends a request directly. A real application must validate the data again on the server.
- Aadhaar and PAN are sensitive personal data. They should be collected only when necessary, handled securely, protected by access controls, and never logged or exposed without need.

15. Future Enhancements

- Show feedback beside each field while the user types.
- Add input masking or spacing (for example 2345 6789 0123) for readability, without changing the underlying value.
- Add server-side validation if the form is connected to a backend.
- Add consent, data minimization, encryption, retention limits and access controls if sensitive data is ever processed.
- Use an authorized verification service, and follow the applicable legal and privacy requirements, wherever real verification is needed.

16. Conclusion

The project combines HTML, CSS, JavaScript and regular expressions in a small form that checks three common Indian input formats and gives immediate feedback. It is a useful exercise in event handling, pattern matching and user feedback, and its code is short enough to read in one sitting. Its results should be taken as format checks only, not as verification of a person's identity or of a document's validity.

17. References

[1] MDN Web Docs, "Regular expressions". https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Regular_expressions

[2] MDN Web Docs, "HTML forms". https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms

[3] MDN Web Docs, "HTMLFormElement: submit event". https://developer.mozilla.org/en-US/docs/Web/API/HTMLFormElement/submit_event